

Optimization Methods for Machine Learning and Engineering

Lecture 15 – Gradient-Free and Non-Convex Optimization

Julius Pfrommer



CC BY-SA 4.0

Agenda

1. Non-Convex Continuous Objective Functions
2. Mixed-Integer Optimization
3. Stochastic Optimization
4. Recap and Outlook

Non-Convex Continuous Objective Functions

Non-Convex Optimization does not scale without further assumptions

- Let $f : [0, 1]^n \rightarrow \mathbb{R}$ a **Lipschitz-continuous function**:

$$\forall \mathbf{x}, \mathbf{y} \in [0, 1]^n : |f(\mathbf{x}) - f(\mathbf{y})| \leq l \|\mathbf{x} - \mathbf{y}\|_2, \quad \text{for a constant } l > 0$$

- The goal is to find a solution $\mathbf{x}$ for which $|f(\mathbf{x}^*) - f(\mathbf{x})| \leq \varepsilon$
- Imagine a function that is completely flat and has only a single “dip” of depth ε .
 - Most feasible solutions have no useful gradient information.
We have to perform a **grid search** to find the “dip”.
 - The **worst-case runtime complexity** for this grid search is $\left\lfloor \frac{L}{2\varepsilon} \right\rfloor^n$.
 - For the case of $n = 16, \varepsilon = 0.01, l = 1$, up to

1.525.878.906.250.000.000.000.000.000

function evaluations have to be performed.

- As a consequence, in the general case, non-convex functions cannot be optimized efficiently.
- We can still solve many non-convex problems
 - by making additional restrictions and limiting assumptions
 - by using **heuristic methods** that may perform in practice even without good theoretical guarantees.

Common Non-Convex Test Functions

Rosenbrock Function (left)

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2$$

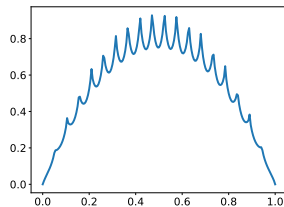
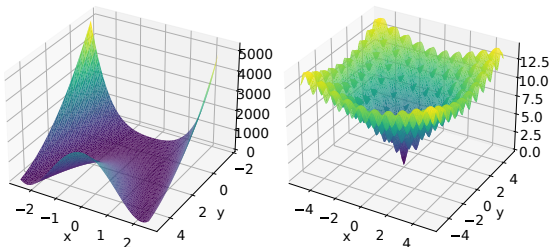
Ackley Function (right)

$$\begin{aligned} f(x, y) = & -20 \exp \left(-0.2 \sqrt{0.5(x^2 + y^2)} \right) \\ & - \exp (0.5(\cos(2\pi x) + \cos(2\pi y))) \\ & + e + 20 \end{aligned}$$

Garland Function (bottom)

$$f(x) = x(1 - x) \cdot (4 - \sqrt{|\sin(60x)|})$$

The Garland Function is particularly nasty. It is not Lipschitz-continuous for any l [Valko2013].



Differential Evolution [Storn1997]

Differential Evolution is a type of **Genetic Algorithms**. Genetic Algorithms are a **nature-inspired heuristics** based on a randomized population of trial solutions that evolves by combination and fitness selection.

Differential Evolution works well for functions that have

- only few non-continuities
- low dimensionality
- box-constraints (known min-max range in all dimensions)

Initialize a population of (thousands of) trial solutions.

Mutate three randomly selected solutions x, y, z into a mutant $v = x + c(y - z)$. Typically $c \in [0.5, 2]$.

Replace the trial solution x with v if $f(v) < f(x)$.

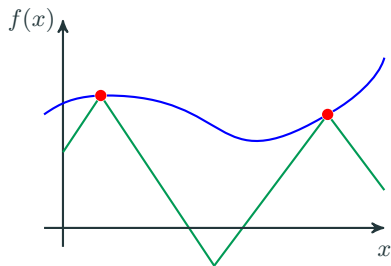
Repeat the mutation and replacement step. After termination return the best-performing trial solution.

An implementation of Differential Evolution for Julia is available via the `EVOLUTIONARY.JL` package. See the implementation here: <https://github.com/wildart/Evolutionary.jl/blob/master/src/de.jl>.

Image Source: Wikipedia

Lipschitzian Optimization

Target Function with Sampled Evaluations (red) and the Lipschitz Lower Bound (green)



The approach was developed independently by [Shubert1972; Piyavskii1972].

If a **Lipschitz constant** l is known for the target function $f : X \rightarrow \mathbb{R}$, the previous function evaluations $\mathbf{y} \in D$ give us a **lower bound** $\underline{f}$:

$$\underline{f}_D(\mathbf{x}) = \min_{\mathbf{y} \in D} [f(\mathbf{y}) - l\|\mathbf{y} - \mathbf{x}\|]$$

For a **compact feasible set** $x \in X$, take the “lowest lower bound” from the Lipschitz constant and evaluate the function at that position. The minimizer of $\underline{f}_D$ can be found by simple geometrical considerations.

But this does not work when the Lipschitz constant is unknown: The target function is a **“black-box”**.

Furthermore, the **theoretical Lipschitz constant is often much larger** than the actual maximum gradient. This leads to a fine-grained grid search where relatively under-performing “regions” are nevertheless dissected into small areas.

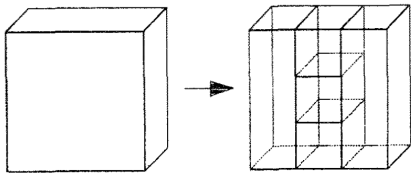
Lipschitzian Optimization without the Lipschitz constant [Jones1993]

Think not of individual evaluation positions, but of **disjoint regions** $r \in R$ from the feasible set. These regions are typically n -dimensional rectangles.

For every region we evaluate the **center-point** x_r to get $f(x_r)$. The **radius of the region** ρ_r is the maximum distance within the region to the center-point.

The regions are successively replaced by **recursively subdividing** them into k more **fine-grained sub-regions**.

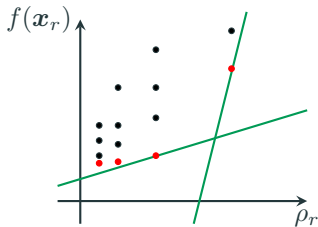
Division of Regions



The Dividing Rectangles (DIRECT) Algorithm

In every iteration, we could select a single region and split it. A logical approach would be to select $\min_{r \in R} [f(x_r) - l \cdot \rho_r]$. But we **do not know the Lipschitz constant l !**

Instead compute a set of **potential regions for any value of l** . Each dot represents a region with its radius and value of the center-point.



Split all potential regions “in parallel” and repeat.

Mixed-Integer Optimization

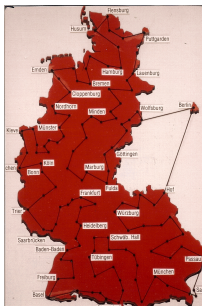
Mixed-Integer Optimization

Instead of $X \subseteq \mathbb{R}^n$ we now **look only at integer solutions** from the feasible set $X \cap \mathbb{Z}^n$.

Integer problems are **combinatorial** and (in general) **much harder** than their continuous counterpart.

The feasible set has no interior $\rightarrow$ Slater's Conditions don't hold $\rightarrow$ **No strong duality**.

Example: Traveling Salesman as a Mixed-Integer Linear Program (MILP) [Dantzig1954; Grötschel1977]



Let $i, j \in N$ cities with d_{ij} distance. Find a tour across all cities of minimum length. Every possible edge $x_{ij} \in \{0, 1\}$ is either taken or not.

This is a MILP $\min_{x \in X} c^\top x$ with travel cost c for possible tours $X = \{0, 1\}^{|N| \cdot (|N|-1)}$. **Relax to an LP** by considering the **convex hull of X** . Additional constraints:

- **Every city** must be connected by **exactly two edges**.
- **No “disconnected subtours.”** Iteratively solve and add constraints to remove subtours: At least two edges must connect the cities in the subtour with the rest.

Job-Shop Scheduling [Manne1960]

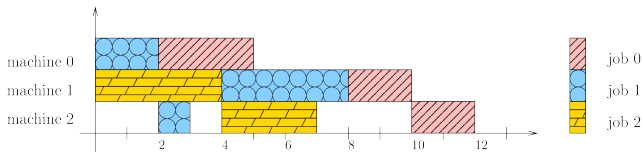
Consider a factory setting with jobs that require time on different machines.

Each job $j \in J$ is a list of operations $(m_o, d_o) \in O$, each occupying a machine m_o for some duration d_o .

$$J = \left\{ \begin{aligned} &[(0, 3), (1, 2), (2, 2)], \\ &[(0, 2), (2, 1), (1, 4)], \\ &[(1, 4), (2, 3)] \end{aligned} \right\}$$

When to process which job to **minimize the time until the last job is done?**

- **The operations for each job need to be processed in order.**
- **Operations cannot be processed on the same machine in parallel.**



The start time of operation o is x_o . The i th operation of job j is $o(j, i)$. The last operation of j has index $j_{\max}$. The integer variable $z_{o,o'} \in \{0, 1\}$ is one if job o' runs on the same machine as o , but at a later time. The scalar $\nu = \sum_{o \in O} d_o$ ensures that later operations don't delay earlier ones.

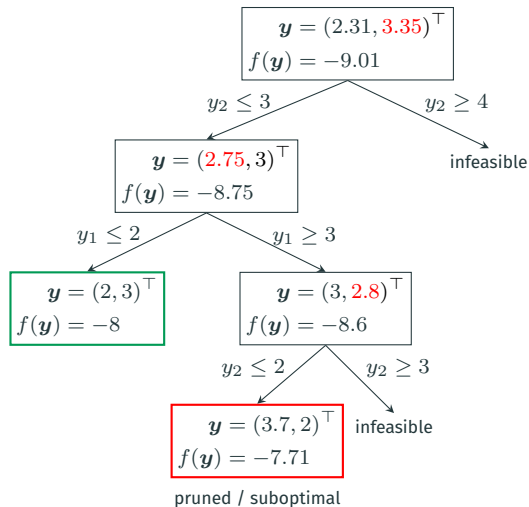
$$\min_{x \in \mathbb{R}_+^{|O|}, z \in \{0,1\}^{|O|^2}} C_{\max}$$

$$\text{subject to } x_{o(j,i)} \geq x_{o(j,i-1)} + d_{o(j,i-1)}, \quad \forall j \in J, \quad i = 2, \dots, j_{\max}$$

$$x_o \geq x_{o'} + d_{o'} - \nu z_{o,o'} \quad \forall o, o' \in O : \\ x_{o'} \geq x_o + d_o - \nu(1 - z_{o,o'}) \quad o < o', m_o = m_{o'}$$

$$C_{\max} \geq x_{o(j,j_{\max})} + d_{o(j,j_{\max})}, \quad \forall j \in J$$

Branch and Bound for Solving MIP [Lawler1966]



Initially, *relax all integer constraints*.

Branch: Afterwards, *iteratively select one variable and split the problem space* by adding constraints that force the variable above/below the integers adjacent to the relaxed solution.

"Most Fractional" Heuristic: Branch on the variable whose fractional part (after the dot) is closest to 0.5.

Bound: *Prune* an entire subtree if the *relaxed solution is worse than a previously known feasible solution*.

$$\begin{aligned} \min_{\mathbf{y} \in \mathbb{Z}_+^2} \quad & f(\mathbf{y}) = -y_1 - 2y_2 \\ \text{subject to} \quad & -\frac{1}{2}y_1 + y_2 \leq \frac{11}{5} \\ & 4y_1 + 5y_2 \leq 26 \\ & 7y_1 + 3y_2 \leq 32 \end{aligned}$$

Stochastic Optimization

Bayesian Optimization

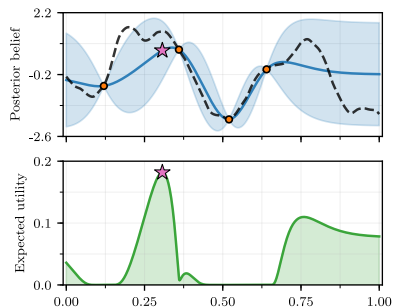
Gaussian Processes (GP) are distributions over functions [Rasmussen2006]. For every position the GP returns a Gaussian.

An **initial prior** is updated via a **kernel function** that defines how observations “from the neighborhood” impact the position under evaluation. Informally, the Kernel defines

- How much we “trust” a function evaluation (how noisy is it).
- How much an evaluation impacts nearby points.

Bayesian Optimization describes the optimization of an initially unknown function by building up a GP that is used to decide on the next evaluation position and updated with the new information.

It is especially used when **evaluations** of the target function are **noisy and expensive**. For example for the optimization of hyper-parameters in machine learning.



Top: Target Function (black), trial evaluations (dots), posterior distribution for the target function (shaded) with its mean (blue).

Bottom: Acquisition function and its optimum.

Image Source: [Wilson2018]

Monte-Carlo Tree Search for Sequential Decision Making [Kocsis2006]

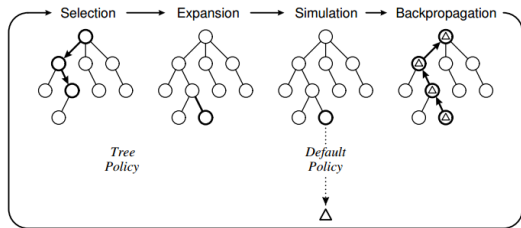


Image Source: [Browne2012]

Nomenclature: **Nodes** i in a tree return a **reward** r_i . Rewards received in the subtree “behind i ” count towards r_i . Each node has been **played** n_i **times**.

Motivation: The Chernoff-Hoeffding Inequality
For n iid realizations x_i drawn from **any random distribution** $x_i \sim \mathbb{P}(\underline{x})$ with mean μ :

$$\mathbb{P}\left(\left(\frac{1}{n} \sum_{i=1}^n x_i - \mu\right) \geq \varepsilon\right) \leq \exp(-2n\varepsilon^2)$$

Start with some initial (randomized) **baseline policy**. This induces an unknown probability distribution for the cumulative reward behind each branch in the tree.

Balance Exploration and Exploitation. That is, find branches with “potential” and explore them further.

For already encountered branches, estimate their value with a **positive exploration bias** for $c > 0$:

$$\bar{r}_i + c\sqrt{\frac{\log(n_{\text{parent}(i)})}{n_i}}$$

Always select the best branch (including the exploration bias). Once arrived at an unexplored branch, perform a “rollout” with the baseline policy.

For every play, **add exactly one new node** at the start of the previously unexplored tree. **Update the empirical mean reward** for all visited nodes.

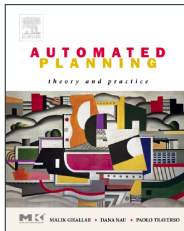
Recap and Outlook

A look back on the semester

1. Introduction, Convexity and Gradient Descent
2. Newton's Method, Loss Functions and Regularization
3. Inequality Constrained Optimization
4. Equality Constrained Optimization
5. Applications: Mechanical Design, Model-Predictive Control, Finance
6. Automatic Differentiation
7. Optimization in Vector Spaces
8. Duality and Primal-Dual Algorithms
9. Support-Vector Machines and the Reproducing Kernel Hilbert Space
10. Conic Programming
11. Gradient-Free and Non-Convex Optimization

Other Technique for Reasoning and Decision Making

Symbolic Planning and Search

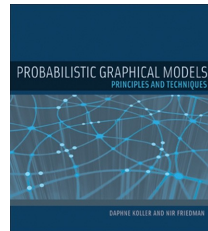
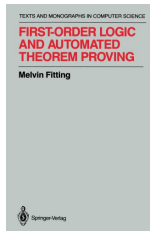


[Ghallab2004; Biere2009]

A **symbolic AI planner** chains together actions (with preconditions and effects) to get from an initial state to a goal state [Fikes1971; Helmert2006].

Satisfiability solvers look for (combinatorial) solutions to a set of constraints. See <https://www.minizinc.org> for a constraint modeling language and solver interface.

Logical and Probabilistic Inference



[Fitting1990; Koller2009]

The breakthrough for **reasoning in first order logic** came with the Unification Algorithm [Robinson1965]. Entire programming languages (e.g. Prolog) are built around logical inference.

Probabilistic inference deals with the efficient representation of and reasoning in large probability distributions [Pearl1988].

That's it for
Optimization Methods for Machine Learning and Engineering!

Thanks to the entire team involved in the development of the course
(from Fraunhofer IOSB, the IES chair and beyond)

Dr. rer. nat. Constanze Hasterok
Josephine Rehak
Alexander Striffler
Jan Philipp Sauer (Video Production)

May you make good use of the techniques from this lecture.
To say it with a quote from Leibniz: ***Calculemus!*** (Let us compute!)

- [Biere2009] Armin Biere, Marijn Heule, and Hans van Maaren, eds. *Handbook of Satisfiability*. IOS press, 2009.
- [Browne2012] Cameron B Browne et al. “A survey of monte carlo tree search methods”. In: *IEEE Transactions on Computational Intelligence and AI in games* 4.1 (2012), pp. 1–43.
- [Barrett2018] Clark Barrett and Cesare Tinelli. “Satisfiability modulo theories”. In: *Handbook of Model Checking*. Springer, 2018, pp. 305–343.
- [Dantzig1954] George Dantzig, Ray Fulkerson, and Selmer Johnson. “Solution of a large-scale traveling-salesman problem”. In: *Journal of the operations research society of America* 2.4 (1954), pp. 393–410.
- [Fitting1990] Melvin Fitting. *First-order logic and automated theorem proving*. Springer, 1990.
- [Fikes1971] Richard E Fikes and Nils J Nilsson. “STRIPS: A new approach to the application of theorem proving to problem solving”. In: *Artificial intelligence* 2.3-4 (1971), pp. 189–208.
- [Ghallab2004] Malik Ghallab, Dana Nau, and Paolo Traverso. *Automated Planning: theory and practice*. Morgan Kaufmann Publishers, 2004.
- [Grötschel1977] Martin Grötschel. “Polyedrische Charakterisierungen kombinatorischer Optimierungsprobleme”. PhD thesis. Universität Bonn, 1977.

- [Helmert2006] Malte Helmert. “The fast downward planning system”. In: *Journal of Artificial Intelligence Research* 26 (2006), pp. 191–246.
- [Jones1993] Donald R Jones, Cary D Perttunen, and Bruce E Stuckman. “Lipschitzian optimization without the Lipschitz constant”. In: *Journal of optimization Theory and Applications* 79.1 (1993), pp. 157–181.
- [Koller2009] Daphne Koller and Nir Friedman. *Probabilistic graphical models: principles and techniques*. MIT press, 2009.
- [Kocsis2006] Levente Kocsis and Csaba Szepesvári. “Bandit based monte-carlo planning”. In: *European conference on machine learning*. Springer. 2006, pp. 282–293.
- [Lawler1966] Eugene L Lawler and David E Wood. “Branch-and-bound methods: A survey”. In: *Operations research* 14.4 (1966), pp. 699–719.
- [Manne1960] Alan S Manne. “On the job-shop scheduling problem”. In: *Operations Research* 8.2 (1960), pp. 219–223.
- [Pearl1988] Judea Pearl. *Probabilistic reasoning in intelligent systems: networks of plausible inference*. Elsevier, 1988.
- [Piyavskii1972] SA Piyavskii. “An algorithm for finding the absolute extremum of a function”. In: *USSR Computational Mathematics and Mathematical Physics* 12.4 (1972), pp. 57–67.

- [Robinson1965] John Alan Robinson. “A machine-oriented logic based on the resolution principle”. In: *Journal of the ACM (JACM)* 12.1 (1965), pp. 23–41.
- [Rasmussen2006] Carl Edward Rasmussen and Christopher Williams. “Gaussian Processes for Machine Learning”. In: MIT Press. 2006.
- [Shubert1972] Bruno O Shubert. “A sequential method seeking the global maximum of a function”. In: *SIAM Journal on Numerical Analysis* 9.3 (1972), pp. 379–388.
- [Storn1997] Rainer Storn and Kenneth Price. “Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces”. In: *Journal of global optimization* 11.4 (1997), pp. 341–359.
- [Valko2013] Michal Valko, Alexandra Carpentier, and Rémi Munos. “Stochastic simultaneous optimistic optimization”. In: *International Conference on Machine Learning*. 2013, pp. 19–27.
- [Wilson2018] JT Wilson, F Hutter, and MP Deisenroth. “Maximizing acquisition functions for Bayesian optimization”. In: *Proceedings of the 32nd Conference on Neural Information Processing Systems (NIPS 2018)*. Neural Information Processing Systems (NIPS). 2018.